

# Práctica 1: Búsqueda

## no informada, informada y con adversarios

Laboratorio de Inteligencia Artificial – Curso 2025-2026

---

Fecha de publicación: 15 de septiembre

Fechas de torneos: 6, 13 y 20 de octubre, a las 20:00.

Entrega: semana del **20 de octubre**, a las 23:59 del día antes de la clase de prácticas

## Introducción

Esta práctica está diseñada para familiarizarse con los principales algoritmos de búsqueda utilizados en inteligencia artificial, tanto en entornos deterministas como en escenarios con adversarios. A lo largo de la práctica, se abordarán técnicas de búsqueda no informada, informada y estrategias competitivas aplicadas a juegos, permitiendo desarrollar agentes capaces de tomar decisiones eficientes. La práctica se divide en dos partes: la primera centrada en la resolución de problemas mediante búsqueda clásica, y la segunda en el diseño de heurísticas y estrategias para búsqueda con adversarios en juegos de dos jugadores. El lenguaje de programación de ambas partes es Python.

## Propuesta de planificación

El enunciado tiene dos partes diferenciadas, ya que los ejercicios a realizar utilizan código diferente. Más aún, en la segunda parte se realiza un torneo, por lo que, se recomienda haber terminado la primera parte antes de que comiencen los torneos. En concreto:

- Semanas del 15 y 22 de septiembre: búsqueda informada y no informada.
- Semanas del 29 de septiembre, 6 y 13 de octubre: búsqueda con adversarios.

## Método de entrega

Las prácticas se harán por parejas. Cada envío debe constar de un único archivo zip llamado **p1\_gggg\_mm\_apellido1\_apellido2.zip**, donde gggg es el número de vuestro grupo de laboratorio y mm el identificador asignado a cada persona/pareja en este grupo. Este archivo zip debe enviarse a través de Moodle.

Es importante, siempre, poner los nombres de los autores y el número del grupo de prácticas en todos los archivos que envíes: memoria y código.

## Material a entregar

El fichero zip a entregar debe contener **dos carpetas** con el mismo nombre que los dos zips que hay en Moodle pero sin la extensión (pacman, juegos). Cada carpeta contendrá:

- El *código* de la práctica, incluyendo **únicamente** los ficheros modificados con vuestro código, tal y como se explica en cada parte del enunciado. En esta carpeta deben aparecer completadas las secciones de código que en la versión entregada al alumno se indican como “***YOUR CODE HERE /TU CÓDIGO AQUÍ***”. No incluyáis ningún otro fichero de código.
- Para la parte A, un fichero *autograder\_results.txt* con el resultado de ejecutar el autocalificador.
- Una memoria de la práctica (en formato PDF)
- Se recomienda que los resultados o comparaciones incluidas en la memoria se muestren **en forma de tablas o gráficas**. Para la parte A, es aconsejable que los razonamientos sobre complejidad no se hagan en base a un solo mapa, sino, en la medida de lo posible, sean *razonamientos generales*, bien aplicando aspectos teóricos o habiendo revisado varios ejemplos de mapas.

**Entre las dos memorias a entregar, no se debe superar el límite máximo de 15 páginas y 4000 palabras (orientativamente, la memoria de la parte A podría tener 8-10 páginas y la de la parte B, 5-7 páginas).**

## Calificación

Cada parte de la práctica se evaluará dentro de un rango de 0-10, donde se tendrá en cuenta tanto la memoria como el código entregado (ver el apartado correspondiente en cada parte). La nota final será el promedio de ambas notas.

**Deshonestidad académica:** Comprobaremos vuestro código con otras entregas en la clase para verificar la redundancia lógica. Si copiáis el código de otra entrega y lo enviáis con cambios menores, lo sabremos. Estos detectores de trampas son bastante difíciles de engañar, así que no lo intentéis. Confiamos en que todos vosotros enviareis únicamente vuestro propio trabajo; *por favor* no nos defraudéis. Si lo hacéis, utilizaremos los recursos disponibles más severos.

**Obtener ayuda:** ¡No estás solo! Si os encontráis atascados en algo, comunicádnoslo a los profesores para poder obtener ayuda.

## PARTE A. Búsqueda no informada e informada

Las tareas a realizar en esta parte de la práctica están relacionadas con la parte del curso dedicada a la búsqueda no informada y búsqueda informada. Para ello utilizaremos una versión modificada del software proporcionado por la Universidad de Berkeley. El software específico para realizar nuestra práctica está en Moodle ([pacman.zip](#)).

### Conceptos básicos

En este proyecto, tu agente Pacman encontrará caminos a través de su mundo laberíntico, tanto para llegar a un lugar en particular como para recolectar comida de manera eficiente. Construirá algoritmos de búsqueda generales y los aplicará a escenarios de Pacman. Este proyecto incluye un autocalificador (autograder) para que califique tus respuestas en la máquina. Este se puede ejecutar con el comando:

```
python autograder.py
```

El código de este proyecto consta de varios archivos de Python, algunos de los cuales deberéis leer y comprender para completar las tareas, y algunos de los cuales podéis ignorar. Ver Apéndice 1 para saber qué hacer con cada fichero.

**Archivos para editar y enviar:** Deberéis completar partes de [search.py](#) y [searchAgents.py](#) durante la tarea. Una vez completada la tarea, incluid estos ficheros junto con el resto para subirlos a Moodle. **No cambiéis el resto de archivos**, ya que aunque los enviéis, no se usarán en la corrección.

**Evaluación del código:** Vuestro código se calificará automáticamente para su corrección técnica. Por favor *no* cambiéis los nombres de las funciones o clases provistas dentro del código, o causará estragos en el autograder. Para la calificación se tendrá en cuenta tanto la **exactitud de la implementación**, como los **resultados** de nuestro autocalificador. En el código tenéis un autocalificador con algunos casos de prueba. Revisaremos y calificaremos las tareas individualmente para asegurarnos de que reciben el debido crédito por su trabajo.

**Calificación:** La nota se dividirá en memoria (40%, incluyendo respuesta a todas las preguntas y las explicaciones que se quieran incluir en cada sección) y código (60%).

# Búsqueda no informada

Después de descargar el código (fichero [pacman.zip](#)), descomprimirlo y cambiar al directorio, deberíais poder jugar un juego de Pacman escribiendo lo siguiente en la línea de comandos:

```
python pacman.py
```

Navegar por el mundo de Pacman de manera eficiente será el primer paso de Pacman para afianzar su dominio. El agente más simple en `searchAgents.py` se llama `GoWestAgent`, que siempre va hacia el oeste (un agente reflejo trivial). Este agente puede ganar ocasionalmente:

```
python pacman.py --layout testMaze --pacman GoWestAgent
```

Pero las cosas se ponen feas para este agente cuando se requiere girar:

```
python pacman.py --layout tinyMaze --pacman GoWestAgent
```

Si Pacman se atasca, podéis salir del juego escribiendo CTRL-C en el terminal. Pronto, vuestro agente resolverá no solo “tinyMaze”, sino cualquier laberinto que desee. El `pacman.py` admite una serie de opciones que pueden expresarse de forma larga (por ejemplo, `--layout`) o corta (por ejemplo, `-l`). Podéis ver la lista de todas las opciones y sus valores predeterminados a través de:

```
python pacman.py -h
```

Además, todos los comandos que aparecen en este proyecto también aparecen en `commands.txt`, para copiar y pegar fácilmente. En UNIX / Mac OS X, incluso podéis ejecutar todos estos comandos en orden con `bash commands.txt`.

Una función útil que puede interesaros es la opción `frameTime` (en segundos), que controla la velocidad de Pacman. Modificarlo puede ayudaros a depurar o simplemente a ver mejor el juego. Por ejemplo:

```
python pacman.py --frameTime 0.5
```

Por otro lado, para depurar python con su depurador podéis escribir la siguiente línea para crear un *breakpoint* en cualquier punto del código:

```
import pdb; pdb.set_trace()
```

## Sección A.1: Encontrar un punto de comida fijo usando la búsqueda primero en profundidad

En `searchAgents.py`, encontraréis un `SearchAgent` completamente implementado, que planifica un camino a través del mundo de Pacman y luego lo ejecuta paso a paso. Además, encontraréis la implementación completa del algoritmo de búsqueda primero en profundidad (DFS). Esta función ya está casi lista para que la probéis en varios laberintos, solo falta implementar una definición para el estado de búsqueda. Así que la primera tarea es completar el código necesario para definir el estado de búsqueda. Podéis probar que el `SearchAgent` funciona ejecutando:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch
```

El comando anterior le dice al `SearchAgent` que use `tinyMazeSearch` como su algoritmo de búsqueda, que está implementado en `search.py`. Pacman debería navegar por el laberinto con éxito.

**Nota importante:** No es necesario construir el árbol de búsqueda de manera explícita. El **estado de búsqueda** (es decir, el nodo en el árbol de búsqueda) debe incluir:

- información sobre la etapa actual en la búsqueda,
- información suficiente para que sea posible tener acceso a la trayectoria desde el principio hasta el final.

Las funciones de búsqueda ya implementadas devolverán la lista de *acciones* que llevan al agente desde el inicio hasta la meta. Estas acciones deben ser movimientos legales (direcciones válidas, no moverse a través de las paredes). En el caso de que la búsqueda falle (el algoritmo no encuentra solución), se debe devolver `None`.

**Nota importante:** Para garantizar que la búsqueda es completa, el algoritmo comprueba si los estados han sido visitados con anterioridad **en el momento en el que son considerados para ser expandidos**, no cuando son generados.

Con respecto a este algoritmo de búsqueda en profundidad (BP) implementado en la función `depthFirstSearch` en `search.py`, responde a las siguientes preguntas:

**Pregunta A.1.1:** ¿Qué significa que sea *completo*?

**Pregunta A.1.2:** ¿Si el algoritmo de BP permitiera expandir nodos ya visitados Pacman siempre puede llegar a la solución? ¿Por qué?

Podéis probar vuestro código con:

```
python pacman.py -l tinyMaze -p SearchAgent
python pacman.py -l mediumMaze -p SearchAgent
python pacman.py -l bigMaze -z .5 -p SearchAgent
```

**\*Recordad** imprimir capturas de pantalla de estos tests, explicar el enfoque empleado, comentar las funciones implementadas/modificadas/usadas y explicar lo que habéis observado para la memoria de la práctica.

El tablero de Pacman mostrará una superposición de los estados explorados y el orden en el que fueron explorados (un rojo más brillante significa que se han explorado antes).

**Pregunta A.1.3:** ¿El orden de exploración es el que esperabais? ¿Pacman *realmente va a todas* las casillas exploradas en su camino hacia la meta?

**Pregunta A.1.4:** ¿La solución que encuentra Pacman al usar BP es la solución de menor coste? ¿Por qué?

*Corrección:* Ejecutad el siguiente comando para ver si vuestra implementación pasa todos los casos de prueba para esta sección de vuestro corrector automático:

```
python autograder.py -q q1
```

## Sección A.2: Algoritmo de búsqueda genérico y búsqueda en anchura

En esta sección, vais a implementar un **algoritmo de búsqueda genérico** (BG) basado en el algoritmo de búsqueda en profundidad (BP) que usasteis anteriormente. El BG *se comportará de forma distinta* dependiendo de la estructura de datos que se le pase como argumento. **Cread la nueva función** `genericSearch` a continuación de `depthFirstSearch`.

**Nota:** Después de implementar el BG, debería ser posible modificar la función `depthFirstSearch` para que únicamente llame al BG pasando la estructura de datos apropiada.

Usando el algoritmo genérico de búsqueda, implementad el algoritmo de búsqueda primero en anchura (BA) en la función `breadthFirstSearch` en `search.py`. Probad vuestro código de forma similar a como lo hicisteis para la búsqueda en profundidad.

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=bfs
python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
python pacman.py -l bigMaze -p SearchAgent -a fn=bfs -z .5
```

**\*Recordad** incluir capturas de pantalla de estos tests, explicar el enfoque empleado, comentar las funciones implementadas/modificadas/usadas y explicar lo que habéis observado para la memoria de la práctica.

**Pregunta A.2.1:** ¿BA encuentra la solución de menor coste? ¿Por qué?

**Pregunta A.2.2:** ¿En el peor caso de BA tendrá que expandir más nodos que en el peor caso de BP? ¿Por qué?

**Pregunta A.2.3:** ¿Por qué pacman cuando usa BA no entra en bucle, aunque haya bucles en el laberinto? ¿Cómo se llama a esta propiedad? *Pista:* el nombre de esta propiedad aparece en las diapositivas de teoría.

*Ayuda:* Si Pacman se mueve demasiado lento, prueba la opción `--frameTime 0`.

*Nota:* Si habéis escrito el código de búsqueda de forma genérica, el código debería funcionar igualmente bien para el problema de búsqueda del ocho puzzle sin ningún cambio.

```
python eightpuzzle.py
```

*Corrección:* Ejecutad el siguiente comando para ver si vuestra implementación pasa todos los casos de prueba de autograder:

```
python autograder.py -q q2
```

## Sección A.3: Variar la función de coste

Si bien BA encontrará un camino con el menor número de acciones hacia la meta, es posible que deseemos encontrar caminos que sean “mejores” en otros sentidos. Considerad `mediumDottedMaze` y `mediumScaryMaze`. Al cambiar la función de coste, podemos alentar a Pacman a encontrar diferentes caminos. Por ejemplo, podemos considerar más costosos pasos peligrosos en áreas plagadas de fantasmas o menos por pasos en áreas ricas en alimentos, y un agente Pacman racional debería ajustar su comportamiento en respuesta.

Implementad el algoritmo de búsqueda en grafo de coste uniforme (BCU) en la función `uniformCostSearch` en `search.py`, para ello, llamad a vuestra función con la implementación del BG pasando la estructura de datos apropiada. Os recomendamos que busquéis en `util.py` la estructura de datos a utilizar. Deberíais observar un comportamiento exitoso en los tres siguientes diseños, donde los agentes utilizados son todos agentes BCU que difieren solo en la función de coste que utilizan (los agentes y las funciones de coste ya se incluyen en el código proporcionado):

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs
python pacman.py -l mediumDottedMaze -p StayEastSearchAgent
python pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```

**\*Recordad** imprimir capturas de pantalla de estos tests, explicar el enfoque empleado, comentar las funciones implementadas/modificadas/usadas y explicar lo que habéis observado para la memoria de la práctica.

*Nota:* Deberíais obtener costes de ruta muy bajos y muy altos para `StayEastSearchAgent` y `StayWestSearchAgent`, respectivamente, debido a sus funciones de coste exponencial (consultad `searchAgents.py` para más detalles).

**Pregunta A.3.1:** Si asignamos el mismo coste a todos los movimientos, ¿BCU expande los mismos nodos que BA?

**Pregunta A.3.2:** ¿Qué coste hay que asignar a cada nuevo nodo en BCU para que se comporte como BP?

*Corrección:* Ejecutad el siguiente comando para ver si vuestra implementación pasa todos los casos de prueba de esta sección en el corrector automático:

```
python autograder.py -q q3
```



# Búsqueda informada

## Sección A.4: Búsqueda A\*

Partiendo de vuestra implementación de `uniformCostSearch`, implementad una búsqueda de grafo A\* en la función vacía `aStarSearch` en `search.py`. A\* toma una función heurística como argumento. La heurística toma dos argumentos: un estado en el problema de búsqueda (el argumento principal) y el problema en sí (como información de referencia). La función heurística `nullHeuristic` en `search.py` es un ejemplo trivial.

Podéis probar vuestra implementación de A\* en el problema original de encontrar un camino a través de un laberinto hasta una posición fija usando la heurística de distancia de Manhattan (implementada ya como `manhattanHeuristic` en `searchAgents.py`).

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

**\*Recordad** imprimir capturas de pantalla de estos tests, explicar el enfoque empleado, comentar las funciones implementadas/modificadas/usadas y explicar lo que habéis observado para la memoria de la práctica.

Deberíais ver que A\* encuentra la solución óptima un poco más rápido que la búsqueda de coste uniforme (aproximadamente 549 frente a 620 nodos de búsqueda expandidos en nuestra implementación, pero los empates en la prioridad pueden hacer que los números difieran ligeramente).

**Pregunta A.4.1:** ¿Qué sucede en `openMaze` para las diversas estrategias de búsqueda?

**Pregunta A.4.2:** ¿Qué cambia en el comportamiento de A\* al usar la distancia de manhattan o la distancia euclídea?

*Corrección:* Ejecutad el siguiente comando para ver si vuestra implementación pasa todos los casos de prueba del corrector automático:

```
python autograder.py -q q4
```

## Sección A.5: Definir un problema de búsqueda

El verdadero poder de A\* solo será evidente con un problema de búsqueda más desafiante. Ahora es el momento de formular un nuevo problema y diseñar una heurística para él (en la siguiente sección).

En *corner mazes*, hay cuatro puntos, uno en cada esquina. Nuestro **nuevo problema de búsqueda** es encontrar el camino más corto a través del laberinto que toca las cuatro esquinas (ya sea que el laberinto tenga comida o no). Tened en cuenta que para algunos laberintos como `tinyCorners`, ¡el camino más corto no siempre va primero a la comida más cercana! *Pista*: el camino más corto a través de `tinyCorners` toma 28 pasos.

*Nota*: Aseguraos de completar la sección A.2 antes de trabajar en esta sección, porque este código se va a basar en vuestra respuesta a dicha sección.

Implementad el problema de búsqueda `CornersProblem` en `searchAgents.py`. Debéis elegir (y **justificar en la memoria**, incluyendo cualquier alternativa que se hubiera descartado) una representación del estado del problema (no confundir con el *nodo de búsqueda*) que codifique toda la información necesaria para detectar si se han alcanzado las cuatro esquinas. Ahora, vuestro agente de búsqueda debería resolver:

```
python pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
python pacman.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

**\*Recordad** imprimir capturas de pantalla de estos tests, explicar el enfoque empleado, comentar las funciones implementadas/modificadas/usadas y explicar lo que habéis observado para la memoria de la práctica.

Para recibir la puntuación máxima, debéis definir una representación de estados abstracta que *no* codifique información irrelevante (como la posición de los fantasmas, dónde está la comida extra, etc.). En particular, no utilizéis un `Pacman GameState` como estado de búsqueda. Vuestro código será extremadamente lento si lo hacéis (y también incorrecto).

*Pista 1*: Las únicas partes del estado del juego a las que debéis hacer referencia en vuestra implementación son la posición inicial de Pacman y la ubicación de las cuatro esquinas.

*Pista 2*: Cuando codifiquéis `getSuccessors`, aseguraos de agregar hijos a la lista de sucesores con un coste de 1. Nuestra implementación de `breadthFirstSearch` expande poco menos de 2000 nodos de búsqueda en `mediumCorners`. Sin embargo, la heurística (utilizada con la búsqueda A\*) puede reducir la cantidad de búsqueda requerida.

*Corrección*: Ejecutad el siguiente comando para ver si vuestra implementación pasa todos los casos de prueba del corrector automático:

```
python autograder.py -q q5
```

## Sección A.6: Heurística para problema de cuatro esquinas

*Nota: Aseguraos de completar las secciones A.4 y A.5 antes de trabajar en esta sección.* Implementad una heurística consistente y no trivial para el CornersProblem en `cornersHeuristic`.

```
python pacman.py -l mediumCorners -p AStarCornersAgent -z 0.5
```

*Nota:* `AStarCornersAgent` es un atajo para

```
-p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=cornersHeuristic
```

**\*Recordad** imprimir capturas de pantalla de estos tests, explicar el enfoque empleado, comentar las funciones implementadas/modificadas/usadas y explicar lo que habéis observado para la memoria de la práctica.

**Admisibilidad frente a consistencia:** Recordad que una heurística es simplemente una función que toma como argumento un estado de búsqueda y devuelve una **estimación** del coste de la trayectoria óptima, la estimación de menor coste, desde ese estado hasta una meta. Las heurísticas más eficaces devolverán valores más cercanos a los costes reales del objetivo. Para ser *admisibile*, las estimaciones proporcionadas por la heurística deben ser cotas inferiores del coste de la trayectoria óptima (de menor coste) hasta una meta. Para ser *consistente*, además debe cumplir que, si una acción tiene un coste  $c$ , realizar esa acción reducirá el coste estimado en  $c$  (como máximo). Recordad que la admisibilidad no es suficiente para garantizar la optimalidad para la búsqueda en grafo; también es necesario cumplir una condición más restrictiva: que la heurística sea consistente (monótona). Afortunadamente, **las heurísticas que se derivan de relajaciones del problema no solo son admisibles, sino que también son consistentes.**

**Nota importante:** La heurística debe ser consistente para cualquier configuración del laberinto, no solo para los laberintos concretos considerados en el autograder proporcionado.

**Nota importante:** Es posible diseñar heurísticas inconsistentes para las cuales el número de nodos expandidos es inferior al obtenido con heurísticas consistentes. Puede que estas respuestas superen los tests del corrector automático. Sin embargo, no son correctas, y recibirán una puntuación inferior a la máxima.

**Heurísticas no triviales:** Las heurísticas triviales son las que

- devuelven cero para todos los estados (BCU),
- computa el coste óptimo para cada uno los estados de búsqueda.

La primera no reduce el coste computacional del algoritmo. Para calcular la segunda, es necesario resolver tantos problemas de búsqueda como estados de búsqueda haya, lo que seguramente provocará un *timeout* en el corrector automático. El objetivo es diseñar una heurística que reduzca el tiempo de cómputo total, que incluye no solo el coste de la búsqueda, sino también el coste del cálculo de la heurística. Busca relajaciones del problema en las cuales los costes de las trayectorias óptimas puedan ser computados de manera eficiente. En esta práctica, el corrector automático solo comprueba el número de

nodos (aparte de imponer un límite en el tiempo de cómputo, de forma que este se mantenga dentro de lo razonable).

**Pregunta A.6.1:** Describe el proceso que habéis seguido para diseñar la heurística y explica la lógica de la misma.

**Calificación:** Vuestra heurística debe ser una heurística consistente, no trivial y no negativa, para recibir puntos. Aseguraos de que vuestra heurística devuelva 0 en cada estado objetivo y nunca devuelva un valor negativo. Dependiendo de la cantidad de nodos que expanda vuestra heurística, se os calificará de acuerdo con el siguiente baremo:

- Más de 2000 nodos expandidos: 0/3
- Como máximo 2000 nodos expandidos: 1/3
- Como máximo 1600 nodos expandidos: 2/3
- Como máximo 1200 nodos expandidos: 3/3

**Recordad:** Si vuestra heurística es inconsistente, vuestra puntuación en esta pregunta será 0, ¡así que tened cuidado!

**Corrección:** Ejecutad el siguiente comando para ver si vuestra implementación pasa todos los casos de prueba en el corrector automático de esta sección:

```
python autograder.py -q q6
```

## PARTE B. Búsqueda con adversarios

A diferencia de la Parte A, que se enfoca en algoritmos de búsqueda clásicos aplicados a entornos estáticos y sin oposición, la Parte B introduce escenarios competitivos donde los agentes deben tomar decisiones en presencia de un adversario. En este contexto, el objetivo no es solo encontrar una solución óptima, sino anticipar y contrarrestar las acciones del oponente. Esta parte de la práctica se centra en el juego *Reversi*, y permite explorar estrategias como *minimax* con poda alfa-beta y el diseño de funciones heurísticas que guíen la toma de decisiones en juegos de dos jugadores, donde la planificación debe considerar tanto el propio beneficio como la posible respuesta del rival. Para ello, se usará el código disponible en Moodle ([juegos.zip](#)) y se realizarán las 3 tareas principales descritas a continuación.

### El juego del Reversi

**Reversi** (también conocido como **Othello** o **Yang**) es un juego de estrategia de dos jugadores. Se juega en un tablero de  $8 \times 8$  con la siguiente configuración inicial:

|   | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |
| 4 |   |   |   | ○ | ● |   |   |   |
| 5 |   |   |   | ● | ○ |   |   |   |
| 6 |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |

Los dos jugadores, representados por las fichas negras y blancas, alternan turnos, empezando las negras. En cada turno, el jugador coloca una pieza de su color sobre el tablero de forma que capture alguna de las fichas del contrario. Se realiza una captura cuando la nueva ficha, junto con alguna o algunas fichas del mismo color en el tablero, encierra hileras de fichas del contrario a lo largo de las filas, las columnas o las diagonales del tablero. Las fichas capturadas son sustituidas por fichas del color del jugador que realiza la captura. En caso de que el jugador que tiene el turno no pueda capturar ninguna ficha del contrario, el turno pasa al otro jugador.

El juego termina cuando ninguno de los jugadores puede capturar fichas del contrario o el tablero está completamente ocupado. El ganador de la partida es el jugador que tiene más fichas en el tablero.

En el siguiente ejemplo el jugador coloca la ficha ● en E6 (izquierda) y captura la ficha ○ que está en E5 (derecha).

|   | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |
| 4 |   |   |   | ○ | ● |   |   |   |
| 5 |   |   |   | ● | ○ |   |   |   |
| 6 |   |   |   |   | ● |   |   |   |
| 7 |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |

|   | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |
| 4 |   |   |   | ○ | ● |   |   |   |
| 5 |   |   |   | ● | ● |   |   |   |
| 6 |   |   |   |   | ● |   |   |   |
| 7 |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |

Podéis jugar online en <https://webgamesonline.com/reversi/> para familiarizados con sus reglas y dinámicas.

## Sección B.1: Búsqueda de soluciones en un entorno estático

Partiendo del juego Reversi que acabamos de explicar, en esta sección vamos a aplicar los métodos de búsqueda implementados en la parte A para encontrar la solución a un problema concreto dado un tablero inicial. Para ello, examina el fichero [othello.py](#) que se encuentra en [pacman.zip](#) y comprueba qué soluciones encuentra al utilizar los distintos métodos (BA, BP, etc.) para varios problemas (OthelloSearchProblemOneCorner, OthelloSearchProblemTwoCorners, OthelloSearchProblemAllCorners).

Ten en cuenta que resolver el juego de esta manera no es realista, ya que asume que el problema es estático y hay una solución óptima; en cambio, los juegos con adversarios no deben solo considerar el entorno sino las posibles acciones del adversario, a poder ser, anticipándose a ellas y respondiendo a las mismas, lo que añade una dimensión estratégica adicional. Es precisamente esto lo que haremos a partir de la próxima sección, pero aprovecha este apartado para comparar hasta dónde se puede llegar con las estrategias de búsqueda clásicas.

En concreto:

**Pregunta B.1.1:** ¿Cómo se comporta cada método de búsqueda con cada uno de los problemas definidos?

**Pregunta B.1.2:** Para el algoritmo A\*, ¿observas alguna diferencia al utilizar las heurísticas incluidas en el fichero?

## Sección B.2: Funciones de evaluación heurística para búsqueda con adversarios

En esta sección, trabajaremos en el diseño y definición de funciones heurísticas que tengan en cuenta el estado del juego y tomen decisiones en función de qué jugador tiene el control en cada momento. Para averiguar su utilidad, las enfrentaremos en varios torneos, para lo cual, deben cumplir una serie de restricciones.

Para probar vuestras funciones de evaluación heurística, debéis crear un fichero con:

1. Tantas clases que hereden de `StudentHeuristic` como funciones de evaluación queráis probar (ver ayuda sobre el código en el Apéndice 2).
2. Dentro de cada clase, tenéis que implementar dos funciones: una de nombre *evaluation\_function* (que recibe un potencial movimiento en la partida, encapsulado como un estado de tipo `TwoPlayerGameState`, y debe devolver una estimación de su interés para el jugador) y otra *get\_name* donde indicaréis el nombre con el que esta heurística aparecerá en el torneo.

A continuación, tenéis un ejemplo con una heurística trivial (no hace nada con el estado), donde podéis ver cómo se pueden incluir funciones auxiliares:

```
from game import (
    TwoPlayerGameState,
)
from tournament import (
    StudentHeuristic,
)

class MySolution1(StudentHeuristic):
    def get_name(self) -> str:
        return "mysolution1"
    def evaluation_function(self, state: TwoPlayerGameState) -> float:
        # let's use an auxiliary function
        aux = self.dummy(123)
        return aux

    def dummy(self, n: int) -> int:
        return n + 4

class MySolution2(StudentHeuristic):
    ...
```

Además de probar las funciones con los programas que se entregan en el proyecto, a través de una interfaz web, podréis subir **el único fichero python** que se os pide para que podáis definir hasta un máximo de 3 estrategias que se usarán en los torneos.

**Importante:** el número de heurísticas para participar en los torneos está limitado a 3, por lo que, aunque incluyáis más clases, no se tendrán en cuenta en el torneo (no debéis asumir que 3 heurísticas se usarán). El nombre de este fichero debe ser único y que identifique a cada pareja: *p1\_gggg\_mm\_apellido1\_apellido2.py*.

Observaciones:

- No se admitirán funciones que hagan llamadas a funciones del sistema, usen funciones de control del juego o manipulen sus estructuras de datos.
- Es importante que la función de evaluación sea eficiente, ya que se controlará el tiempo invertido por cada jugador, de manera que se dará la partida por perdida a aquel jugador que sea muy lento. Este baremo de cálculo no podrá ser un tiempo absoluto (que depende, además de la máquina, de diversos factores, entre ellos del SO utilizado y de la profundidad de la búsqueda). No obstante, para dar una indicación que sea reproducible en cualquier máquina, habrá que tener en cuenta que **aquellos jugadores que tarden más que 10 veces lo que tarda la heurística trivial** (clase `Heuristic1` en `demo_tournament.py`) en un tablero 8x8 muy probablemente serán eliminados por *timeout*. No es posible asegurar que siempre serán eliminados, porque la duración de una evaluación depende también del nº de movimientos posibles, lo que introduce cierta variabilidad.



## Sección B.3: Minimax con poda

Se proporciona una implementación del algoritmo *minimax* (ver clase MinimaxStrategy en el módulo strategy.py). Esta implementación está basada en el siguiente pseudocódigo:

```
function MINIMAX-DECISION(state) returns an action
  inputs: state, current state in game
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$ 
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow \infty$ 
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$ 
  return v
```

Para realizar esta parte de la práctica, define una clase de nombre MinimaxAlphaBetaStrategy que herede de Strategy (puedes utilizar la clase MinimaxStrategy como punto de partida) que implemente el algoritmo minimax con poda alfa-beta. La implementación debe ser acorde con este pseudocódigo:

```
function ALPHA-BETA-DECISION(state) returns an action
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
          $\alpha$ , the value of the best alternative for MAX along the path to state
          $\beta$ , the value of the best alternative for MIN along the path to state
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for a, s in SUCCESSORS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$ 
    if  $v \geq \beta$  then return v
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  return v

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  same as MAX-VALUE but with roles of  $\alpha$ ,  $\beta$  reversed
```

# Evaluación de la Parte B

Para la calificación de esta parte (evaluada en rango 0-10), se tendrán en cuenta los siguientes 3 aspectos.

## Calificación de funciones de evaluación heurística (3 puntos)

Consiste en la implementación y presentación de funciones de evaluación heurística a un torneo entre todas las parejas de estudiantes de la asignatura. Los estudiantes dispondrán de 10 tokens al inicio de la práctica, donde cada ejecución de un torneo interno consumirá un token. Estos torneos internos permitirán averiguar cómo se comportan las heurísticas implementadas frente a funciones heurísticas de varios niveles provistas por los profesores. En los torneos se probarán **tableros 8x8** comenzando con el estado inicial por defecto y a **profundidades de minimax no mayores de 4**, por lo que es recomendable realizar pruebas con esta configuración.

### Torneo general (1.5 puntos)

Como resultado de los enfrentamientos entre funciones de evaluación se publicará una clasificación, accesible mediante un enlace que se mostrará en la parte del torneo del curso Moodle. En las fechas señaladas más abajo, se realizará un torneo entre las estrategias que estén subidas y se actualizará el ranking en la página de Moodle habilitada para ello. En cierto momento se introducirán las heurísticas de referencia proporcionadas por los profesores, las cuales corresponden a las de los torneos internos y servirán de indicadores de nivel de juego.

Se recomienda a los estudiantes ejecutar torneos internos contra las heurísticas de referencia de los profesores (usando los tokens que tengan disponibles) antes de que se realicen los torneos generales, para obtener una estimación del rendimiento de sus propias heurísticas. Estos torneos se explican en la siguiente sección.

La nota de esta parte se calculará de la siguiente manera:

| Nota obtenida | Percentil X en el ranking |
|---------------|---------------------------|
| 150,00 %      | $X \geq 90$               |
| 125,00 %      | -                         |
| 100,00 %      | $90 > X \geq 50$          |
| 75,00 %       | $50 > X \geq 10$          |
| 50,00 %       | $10 > X$                  |
| 25,00 %       | -                         |
| 0,00 %        | -                         |

Cada torneo tendrá un peso que se irá incrementando: **15, 35 y 50%**.

Habrà torneos abiertos los siguientes días, antes de cuya fecha se tendrá que subir 3 funciones (al menos una DEBE ser distinta a las enviadas anteriormente o habrá una penalización):

- **Lunes 6 de octubre** (subir antes de las 20:00)
- **Lunes 13 de octubre** (subir antes de las 20:00)

Estas dos entregas son obligatorias y **será penalizada la no entrega** (con un 0 en el ranking correspondiente por cada entrega que falte). La entrega final de funciones de evaluación será el día:

- Lunes **20 de octubre** (subir antes de las 20:00).

La participación a los torneos se hace separada de la entrega completa de la P1 (la cual se hace por Moodle) en las fechas indicadas al comienzo de este enunciado. Para los torneos se usará la interfaz web disponible en

<http://150.244.56.40:5000/>

Para entrar a este sistema, deberás usar tu dirección de correo electrónico y una contraseña que podrás encontrar en la parte de retroalimentación de la tarea de Moodle con nombre “Acceso para usar sistema de torneo en la P1”.

**Importante:** los envíos están controlados por parejas de prácticas, es decir, que los dos miembros de la pareja participan con el mismo envío de heurística, por lo que es crítico que el sistema tenga esta información actualizada. Si el sistema muestra a los dos miembros de la pareja información diferente sobre su grupo de prácticas o número de pareja, informad al coordinador de prácticas.

En cualquier caso, tanto en el código como en la memoria entregados se volverán a incluir las estrategias seleccionadas y se analizarán en detalle las distintas soluciones presentadas, incluyendo referencias a artículos (siguiendo el formato APA<sup>1</sup>) que expliquen o justifiquen las heurísticas planteadas.

### Torneo interno.

Los estudiantes pueden ejecutar torneos internos utilizando los tokens asignados al inicio de la práctica. Cada ejecución de un torneo interno consume un token. Hay un total de 10 tokens disponibles por pareja al inicio de la práctica. En los torneos internos, las heurísticas entregadas por los estudiantes se enfrentarán a las heurísticas de referencia proporcionadas por los profesores, que también participan en los torneos generales, y a una heurística random (ver siguiente apartado). Un uso estratégico de los tokens permitirá obtener información valiosa para ajustar y mejorar las heurísticas antes de competir en los torneos generales.

### Enfrentamientos con random (1.5 puntos)

Las tres heurísticas entregadas en el **último torneo** se enfrentarán 1000 veces contra una heurística trivial que asigna valores aleatorios. Estos enfrentamientos se realizarán utilizando minimax con una profundidad de 1. La nota asociada a estos enfrentamientos se calculará de la siguiente manera:

---

<sup>1</sup> APA Quick Citation Guide (visitada por última vez en febrero 2022):

<https://guides.libraries.psu.edu/apaquickguide/intext>.

Observación: se pueden obtener referencias en formato APA muy fácilmente buscando el artículo en cuestión en Google Scholar, y luego pulsando en el botón de “Cite”. Por ejemplo, al hacer eso con la siguiente URL: <https://scholar.google.com/scholar?q=the%20texbook>, eligiendo el formato APA nos devuelve:

Knuth, D. E., & Bibby, D. (1984). *The texbook* (Vol. 15). Reading: Addison-Wesley.

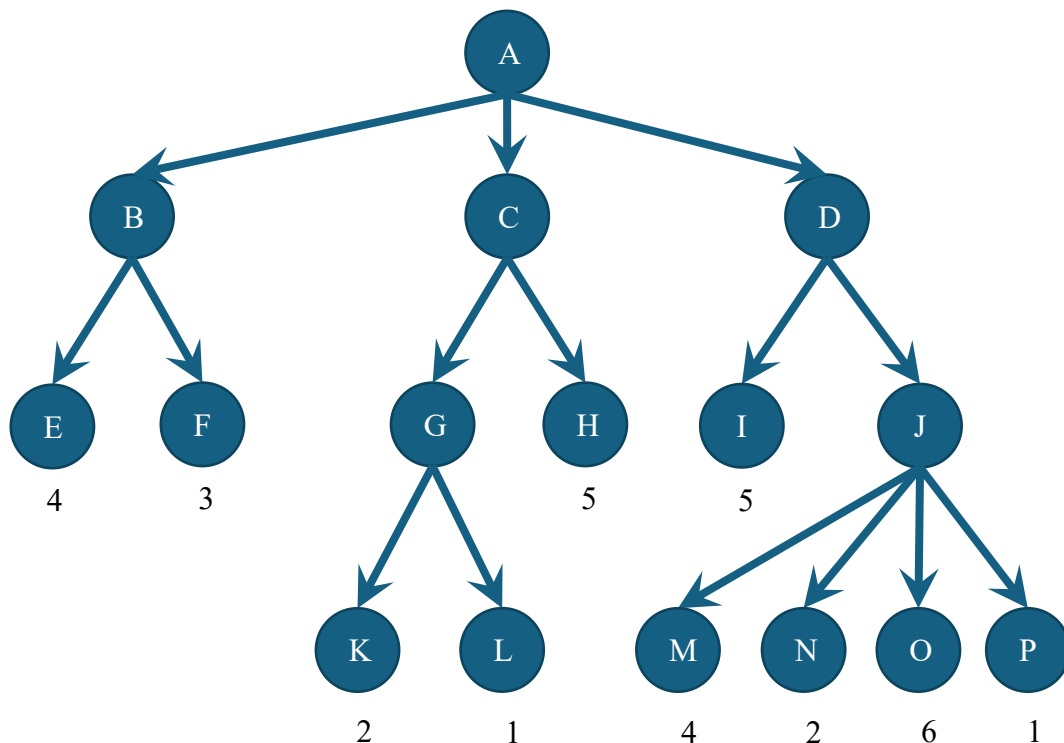
$$nota = 1.5 \cdot \max\left(0, \left(\frac{\# \text{victorias}}{1000} - 0.5\right) \cdot 2\right)$$

Se recomienda comprobar el comportamiento de las heurísticas desarrolladas con una heurística trivial, ya que ganar un 50% o menos de las veces contra ella indica que vuestra heurística toma peores decisiones que si simplemente evaluara los tableros al azar.

## Calificación de algoritmo minimax con poda (3 puntos)

El código entregado se valorará tanto por funcionalidad como por estilo. Es importante documentar en la memoria de forma clara el algoritmo empleado en la función realizada, así como su funcionamiento y sus características más relevantes. Para justificar que la poda está funcionando, se debe entregar un informe de tiempos, para lo cual, se aconseja el uso de la librería `timeit`<sup>2</sup>. Para ser más específicos y evitar variaciones debidas a jugadores no deterministas, se realizará el análisis con un jugador que utilice la estrategia determinista mencionada en la Sección B.2, es decir, *Heuristic1* en `demo_tournament.py` a profundidades 3 y 4. Además, deberías realizar el mismo análisis con una heurística tuya y comparar los resultados, incluyendo la discusión en la memoria. **Ten en cuenta que los tiempos pueden depender del ordenador en el que trabajas, por lo que, además de los tiempos absolutos, deberíais proporcionar alguna medida de la mejora que sea independiente de dicho ordenador.**

Con el fin de determinar si las implementaciones de minimax y minimax con poda alfa-beta son correctas, se propone considerar el juego definido por el siguiente árbol:



La especificación del juego se encuentra en el fichero `simple_game_tree.py`.

<sup>2</sup>Documentación de `timeit` (visitada por última vez en Septiembre 2024):  
<https://docs.python.org/3/library/timeit.html>

Para este juego:

- (i) Aplica manualmente el algoritmo minimax. Sobre una copia del árbol, identifica el número de iteración, el nodo que se está visitando y la estimación del valor minimax para dicho nodo en cada paso del algoritmo.
- (ii) Comprueba que la ejecución manual coincide con la realizada por el jugador minimax en demo\_simple\_game\_tree.py que inicia la partida. Para ello, indica en la memoria la información correspondiente al primer movimiento del juego con `verbose=3`.
- (iii) Aplica manualmente el algoritmo minimax con poda alfa-beta. Identifica el número de iteración, el nodo que se está visitando y la estimación del intervalo  $[a, b]$  para dicho nodo en cada paso del algoritmo. Finalmente, indica el valor minimax en el nodo raíz y la secuencia de movimientos hasta el final de la partida.
- (iv) Comprueba que la ejecución manual coincide con la realizada por el jugador minimax que utiliza poda alfa-beta en demo\_simple\_game\_tree.py. que inicia la partida. Para ello, **indica en la memoria la secuencia de valores del intervalo  $[a, b]$  asociados a los nodos visitados en cada iteración del algoritmo.**

## Memoria (4 puntos)

Consiste en una memoria con una descripción de las funciones de evaluación implementadas, así como la documentación asociada a la implementación del algoritmo alfa-beta, que se subirá a Moodle junto con el código de las otras partes. **El informe debe ser claro, completo y conciso. Por favor, incluye sólo información relevante.**

1. Documentación de minimax y la poda alfa-beta
  - a. Detalles de implementación: ¿qué tests se han diseñado y aplicado para determinar si la implementación es correcta?, diseño e implementación (estructuras de datos seleccionadas, descomposición funcional, etc.), otra información relevante.
  - b. Eficiencia de la poda alfa-beta: descripción completa del protocolo de evaluación, tablas donde se incluyan tiempos con y sin poda, medidas de mejora independientes del ordenador, análisis correcto, completo y claro de los resultados, otra información relevante.
2. Documentación del diseño de la heurística.
  - a. Revisión de trabajos previos sobre estrategias de Reversi, incluyendo referencias en el formato APA.
  - b. Descripción del proceso de diseño: ¿cómo se planeó y ejecutó el proceso de diseño? ¿seguiste algún procedimiento sistemático para evaluar las heurísticas diseñadas? ¿cuál fue el rendimiento de tus heurísticas contra las heurísticas de referencia? ¿y contra random? ¿te apoyaste en esos resultados para plantear modificaciones? ¿utilizaste ideas desarrolladas por otros para mejorar las estrategias diseñadas? Si están disponibles públicamente, incluye referencias en formato APA; en otro caso, incluye el nombre de la persona que te dio la información y dale el crédito oportuno como “comunicación privada”.
  - c. Descripción de la heurística enviada finalmente.
  - d. Otra información relevante.

## Apéndice 1: Código para parte A de la práctica

El código de este proyecto consta de los siguientes archivos de Python que se incluyen en la tabla, donde se indica cuales deberéis leer y comprender para completar las tareas, y cuáles podréis ignorar.

- Ficheros a editar:
  - `search.py` Donde residirán todos vuestros algoritmos de búsqueda.
  - `searchAgents.py` Donde residirán todos vuestros agentes basados en búsquedas.
- Archivos a usar en la parte B de la práctica:
  - `othello.py` Código similar a `eightpuzzle` pero que resuelve tableros de Othello usando tus funciones de búsqueda.
- Archivos que *quizás* queráis ver:
  - `pacman.py` El archivo principal que ejecuta los juegos de Pacman. Este archivo describe un tipo `GameState` para Pacman, que se usa en este proyecto.
  - `game.py` La lógica detrás de cómo funciona el mundo Pacman. Este archivo describe varios tipos de soporte como `AgentState`, `Agent`, `Direction` y `Grid`
  - `util.py` Estructuras de datos útiles para implementar algoritmos de búsqueda.
  - `test_cases/` Directorio que contiene los casos de prueba para cada pregunta.
- Archivos de apoyo que podéis ignorar:
  - `graphicsDisplay.py` Gráficos para Pacman.
  - `graphicsUtils.py` Soporte para gráficos Pacman.
  - `textDisplay.py` Gráficos ASCII para Pacman.
  - `eightpuzzle.py` Código que permite usar tus funciones de búsqueda para resolver el problema del ocho puzzle.
  - `ghostAgents.py` Agentes para controlar fantasmas.
  - `keyboardAgents.py` Interfaces de teclado para controlar Pacman.
  - `layout.py` Código para leer archivos de diseño y almacenar su contenido.
  - `autograder.py` El calificador automático.
  - `testParser.py` Analiza los archivos de prueba y solución del autograder.
  - `testClasses.py` Clases generales de prueba de calificación automática.
  - `searchTestClasses.py` Clases de prueba de autocalificación específicas para esta práctica.

## Apéndice 2: Código para parte B de la práctica

El código proporcionado requiere la versión 3.7 de Python, aunque recomendamos usar la 3.10. La infraestructura para un juego entre dos jugadores está en el módulo [game.py](#). Las clases y funciones específicas para Reversi están implementadas en el fichero [reversi.py](#). **Estos ficheros no deben ser modificados.**

Las estrategias para el juego (jugador aleatorio, jugador manual, jugador minimax) están implementadas en [strategy.py](#). **Este es el fichero que se debe modificar para implementar un jugador que realice una búsqueda minimax con poda alfa-beta (ver parte 1 de la evaluación).**

En el módulo [heuristic.py](#) se ha implementado un ejemplo de función de evaluación de estados del juego, la cual es utilizada en estrategias de búsqueda. Esta función es general al no usar conocimiento del juego en cuestión, por lo que tiene bastante margen de mejora, sin embargo, por ello mismo se puede usar la misma heurística para varios juegos.

Podéis encontrar ejemplos de cómo jugar en el fichero [demo\\_reversi.py](#).

En los distintos ejemplos que podéis ver en este fichero, se observa lo siguiente:

- La lógica del juego (en este caso, Reversi) se abstrae dentro de una clase de tipo `TwoPlayerGameState`, que almacena la información necesaria para juegos de dos jugadores (además de la lógica del juego, el tablero, el jugador inicial, si se ha terminado el juego, las puntuaciones, etc.)
- La lógica de una partida de dos jugadores se maneja a través de la clase `TwoPlayerMatch`, que recibe un estado de tipo `TwoPlayerGameState` y es el encargado de dar turno a cada jugador, imprimir más o menos información por pantalla (según el valor de la variable `verbose`), comprobar que ningún jugador invierta mucho tiempo en cada turno (ver variable `max_seconds_per_move`), así como de controlar si la interfaz será textual o gráfica (variable `gui`).

Para terminar de entender el código entregado, se explican a continuación las clases y funcionalidades más importantes:

- El parámetro `gui` de la clase `TwoPlayerMatch` sirve para decidir si el juego se mostrará como texto en la terminal o si se creará una ventana gráfica. En cualquiera de los dos casos se pueden usar cualquiera de los tipos de jugadores (manual o automático). Si el jugador es manual, el comportamiento es diferente: en el caso de que se haya elegido jugar con la interfaz gráfica (el valor de la variable `gui` es `True`), entonces el jugador tendrá que hacer clic en la ventana; en modo texto (`gui=False`) realizará su jugada por la terminal.
- La variable `verbose > 0` permite obtener información sobre la búsqueda minimax. En concreto,
  - `verbose = 1` # Se imprime el valor minimax de la jugada seleccionada
  - `verbose = 2` # Se ejecuta el juego paso a paso y se imprimen los valores minimax de todos los estados explorados en la búsqueda.
  - `verbose = 3` # Se ejecuta el juego de manera continua y se imprimen los valores minimax de todos los estados explorados en la búsqueda.

Adicionalmente, en los modos con  $0 < \text{verbose} < 3$  se ofrece la posibilidad de almacenar el estado intermedio correspondiente; es decir la información sobre el jugador que va a realizar el movimiento y el estado del tablero.

- En [demo\\_reversi.py](#) podéis encontrar distintos jugadores inicializados: manuales, aleatorios, que utilizan estrategias con búsqueda (aplicando el algoritmo minimax para la búsqueda y una función para la evolución de estados del juego). Los jugadores se implementan como estrategias de juego (clase abstracta Strategy definida en el módulo [strategy.py](#)). También se puede ver cómo crear un jugador que utilice una heurística creada por vosotros.
- Podéis también explorar otros juegos más simples (puede ser útil para depurar el código): [demo\\_tictactoe.py](#) y [demo\\_simple\\_game\\_tree.py](#).
- En el módulo [heuristic.py](#) se incluye una función heurística (simple\_evaluation\_function) que se puede pasar a jugadores de tipo minimax. En el siguiente apartado veréis que **uno de vuestros objetivos en esta práctica es diseñar una función heurística que sea competitiva**.
- El tipo de estrategia MinimaxStrategy, además de la función heurística, recibe como argumento la máxima profundidad a la que se hará la búsqueda (max\_depth\_minimax).
- Para permitir experimentar más rápido y probar distintas situaciones, la infraestructura entregada permite jugar a partir de un estado intermedio. Para ello es necesario inicializar las variables initial\_board e initial\_player con los valores correspondientes. Para jugar a partir de la configuración estándar se puede utilizar board = None en la instanciación de un objeto de tipo TwoPlayerGameState.
- Las funciones from\_dictionary\_to\_array\_board, from\_array\_to\_dictionary\_board que se encuentran en el fichero reversi.py permiten transformar el tablero en forma de diccionario (el usado en la implementación del juego) a uno en forma de array (más fácil de visualizar), y viceversa.
- Los ficheros [tictactoe.py](#) y [demo\\_tictactoe.py](#) permiten jugar al Tres en Raya, un juego más sencillo que Reversi y con menor factor de ramificación, que puede ser útil para vuestras pruebas.
- Para la implementación del algoritmo minimax con poda alfa-beta, puede ser útil implementar un juego más sencillo o analizar alguno de los que se dan en la práctica. Por ejemplo, uno definido por un árbol de juego (ver [simple\\_game\\_tree.py](#) y [demo\\_simple\\_game\\_tree.py](#)).

Además de probar a jugar las partidas que están configuradas en el módulo [demo\\_reversi.py](#), también podéis llevar a cabo un torneo con varias estrategias y funciones de evaluación llamando a la función run del módulo tournament.py indicando:

1. Las estrategias.
2. El número de partidas con fichas negras (se jugará el mismo número con blancas).
3. Los nombres o alias de las estrategias.



Una posible salida del código que tenéis en [demo\\_tournament.py](#) es la siguiente:

|                | total: | opt1_dummy | opt2_random | opt3_heuristic |
|----------------|--------|------------|-------------|----------------|
| opt1_dummy     | 2:     | ---        | 2           | 0              |
| opt2_random    | 9:     | 8          | ---         | 1              |
| opt3_heuristic | 19:    | 10         | 9           | ---            |

En esta configuración se han jugado 30 partidas en total (20 para cada estrategia):  $30 = 5 \times 2 \times 3 = 5$  repeticiones (parámetro `n_pairs`)  $\times$  2 jugadores (cada estrategia juega como blancas y negras)  $\times$  número de estrategias (3 en este caso). Se puede ver que la estrategia random gana 9 de los 30 juegos, 8 contra dummy y 1 contra heuristic.

En el mismo fichero podéis ver (entre comentarios) cómo cargar estrategias desde un fichero que exista en una carpeta, tal y como se cargarán en el torneo. Puede ser útil probarlo de esta manera para confirmar que funciona sin problemas.

**Observación:** la función `run` del módulo [tournament.py](#) usa `MinimaxStrategy`, sin embargo, cuando tengáis implementado `MinimaxAlphaBetaStrategy`, es aconsejable que lo cambiéis para ahorrar tiempo en vuestras ejecuciones.